

Revisione Sprint 0

Corso di Ingegneria dei Sistemi Software M

08 Luglio 2026

1 I Fondamenti dello Sprint nell'Ingegneria del Software

- **Goal Esplicito:** Ogni singolo sprint deve definire in modo univoco ed esplicito il proprio obiettivo. Non sono accettabili sprint privi di un focus chiaro e dichiarativo.
- **Producibile Concreto (Software Funzionante):** Poiché ogni sprint ha un fine prestabilito, deve necessariamente rilasciare un deliverable tangibile ed effettivo. In termini ingegneristici, ciò si traduce nel **costruire software effettivo, eseguibile e comprensibile alla macchina**. Se uno sprint (a partire dallo Sprint 0) non costruisce o non produce nulla verificabile dall'elaboratore, non è metodologicamente ammissibile.

2 Lo Sprint 0 e l'Analisi dei Requisiti

Nel contesto dello **Sprint 0**, dedicato esclusivamente all'analisi dei requisiti, l'artefatto concreto interpretabile dall'elaboratore consiste nella formalizzazione delle specifiche tramite un **modello eseguibile (file QAK)**, affiancato dalle definizioni delle **interfacce** e dal **test plan** iniziale. Di fronte alla macchina, i file QAK derivanti dall'analisi dei requisiti dovrebbero risultare equivalenti e coerenti tra loro, dicendo la stessa cosa all'elaboratore.

2.1 Il ruolo del Committente e l'Interazione

In questa fase è imprescindibile il confronto con un attore fondamentale esterno al team: **il Committente**. Lo scopo principale di questa interazione consiste nel porre domande mirate per risolvere le ambiguità, le lacune e i punti oscuri del testo dei requisiti originario, sfruttando il feedback per produrre specifiche esatte.

2.2 Costruire vs. Esprimere

- **Costruire:** È l'atto tecnico dell'implementazione pura di una funzionalità o di un componente. Può essere fatto in qualsiasi linguaggio di programmazione di uso generale. Alcuni linguaggi storici (es. il C rispetto all'Assembly) possiedono capacità strutturali migliori, ma rimangono legati alla computazione locale.
- **Esprimere:** È un concetto semantico di livello superiore. Significa dichiarare la natura intrinseca e i vincoli architetturali di un componente in modo nativo affinché la macchina li comprenda immediatamente (ad esempio, dichiarare esplicitamente che un componente è un *servizio autonomo remoto* e non un semplice oggetto locale).

2.3 I limiti espressivi dei linguaggi tradizionali e la transizione al Modello

In un linguaggio come Java non esiste la parola chiave nativa **service**, ma esiste solo **class**. Di conseguenza, in Java i servizi si possono *costruire* (tramite design pattern, configurazioni di rete, socket o framework), ma non si possono *esprimere*.

Davanti a questa carenza di capacità espressiva del linguaggio di programmazione, si avverte la necessità di passare a un **Linguaggio di Modellazione (QAK)** supportato da una **Software Factory**, introducendo un’innovazione tecnologica e concettuale (un atto di invenzione nativa che un’IA non è in grado di replicare autonomamente).

- **Programma:** Un insieme di istruzioni scritte in un linguaggio che si concentra sul *come* implementare i dettagli tecnologici di basso livello.
- **Modello:** Una rappresentazione formale del sistema orientata al *cosa*, in cui le classi astratte vengono elevate a entità di prima classe direttamente comprensibili dall’infrastruttura.

3 Dimensione Architetture e Analisi del Dominio

L’analisi dei requisiti iniziale impone tre risposte fondamentali per definire la topologia del sistema. Il linguaggio QAK nasce specificamente per modellare software distribuito.

Domanda Chiave	Valutazione Architetture
Software distribuito o concentrato?	Distribuito. Concentrato significa che tutto l’eseguibile risiede su un unico nodo; distribuito implica che il software è suddiviso su nodi computazionali diversi che comunicano via rete.
Sistema omogeneo o eterogeneo?	Eterogeneo. I requisiti inducono l’adozione di linguaggi di programmazione e tecnologie differenti sui vari nodi.
Numero di nodi richiesti?	4 nodi computazionali indipendenti sulla base del partizionamento delle entità.

3.1 Partizionamento delle Entità nei Nodi

Il testo dei requisiti evoca diverse entità che devono essere ripartite e mappate sui rispettivi contesti fisici o logici (*Context*):

- **Nodo CargoService:** Ospita la logica di business centrale. Contiene la rappresentazione della *hold* e dello *slot*. Nota bene: I *container* fisici evocati nel testo non vanno espressi come entità software autonome.
- **Nodo PicoW:** Gestisce le periferiche hardware e l’I/O ravvicinato del microcontrollore Raspberry Pi Pico W, nello specifico il *sonar* e il *led*.
- **Nodo IOPort:** Definito formalmente come un *device*. Gestisce l’interfaccia utente e ospita i componenti fisici o logici *pushbutton* e *display*.
- **Nodo CargoRobot:** Richiede un contesto (*Context*) separato e un nodo differente rispetto a *CargoService*, in quanto ente deputato alla gestione del robot di trasporto.

4 Focus: CargoRobot — POJO o Servizio?

Il *CargoRobot* deve obbedire ai comandi impartiti dal *CargoService*. Dai requisiti emerge come un ente non autonomo, in costante attesa di richieste. La scelta della sua natura software è legata al concetto di **Interazione (Interaction)**:

- **POJO (Procedure Call):** L'interazione avviene tramite chiamata di procedura. Il chiamante (A) trasferisce rigidamente il controllo del flusso a B (*CargoRobot*); il modulo A si disattiva ("*diventa morto*") rimanendo bloccato finché B non restituisce il controllo tramite una *return*. Il POJO vive solo nello spazio di memoria e nel thread del chiamante sulla stessa macchina.
- **Servizio (Message-driven / Autonomo):** È un'entità remota dotata di un proprio thread di controllo. Per utilizzarlo è necessario connettersi conoscendone l'IP e i protocolli di comunicazione, interagendo via messaggi asincroni.

4.1 Il superamento della OOP a favore dei Microservizi

Nel panorama software contemporaneo le grandi software house hanno superato il paradigma Object-Oriented (OOP) puro basato su POJO a favore dell'architettura a **microservizi**. Il guadagno strategico è legato all'interazione a messaggi: modellando il robot come un servizio, lo si può scrivere con qualsiasi linguaggio di programmazione, lo si alloca ovunque nella rete e lo si fa girare liberamente senza bloccare l'esecuzione o il flusso del *CargoService*.

4.2 Analisi delle soluzioni esistenti ed esplorazione del dominio

Prima dell'analisi è cruciale verificare l'esistenza di componenti preesistenti. Il committente ha chiarito che il robot fisico/virtuale esiste già sotto forma di *virtualrobot26*. L'ecosistema mette a disposizione quattro varianti d'integrazione (tre servizi e un POJO); la motivazione della scelta tecnologica della variante corretta costituirà il fulcro dello Sprint 1.

4.3 Vincolo metodologico fondamentale dello Sprint 0

Non è ammesso inserire nello Sprint 0 i comandi operativi dettagliati del robot o gli stati del LED (es. comandi di movimento o messaggi specifici). Tali dettagli non sono esplicitati nei requisiti di partenza del committente; inserirli in questa fase significherebbe inventare specifiche arbitrarie. Lo Sprint 0 mappa l'esistenza e la natura delle entità; l'interazione dinamica dettagliata appartiene allo **Sprint 1**, il quale guiderà lo sviluppo fino a un primo prototipo completo ed eseguibile, validato dal relativo *Test Plan*.

5 La Natura di QAK: Il significato di "Quasi" Attore

Un **QActor** può essere espresso nativamente come un servizio, un microservizio o un nanoservizio. La lettera "**Q**" (*Quasi*) racchiude una proprietà architetturale fondamentale:

Durante la fase di sviluppo, un componente può essere mantenuto momentaneamente all'interno di un unico contesto locale insieme ad altri attori coordinati (comportandosi da nanoservizio). Tuttavia, una volta completato e validato lo sviluppo, tale componente può essere isolato e spostato all'esterno su un nodo remoto di rete (es. un *led mock*) **senza dover modificare una singola riga del suo codice sorgente**.

6 Chiarimenti Specifici del Committente

- **IOPORT:** Formalizzato come *device*. Il committente richiede espressamente lo sviluppo di una **Web-GUI** di controllo che esponga visivamente un pulsante (*button*) e un display.
- **LED:** Rappresenta la luce di stato fisica integrata o connessa direttamente al microcontrollore Raspberry Pi Pico W (*PicoW*), lo stesso nodo a cui è cablato il sensore *Sonar*.